Challenge 2: Log Forensics

Hrichik Khandait | hrichik2006@gmail.com

03 October 2026

1. Executive Summary

This report presents the forensic review of agent logs collected from Organisation X, an environment of roughly 250 Linux desktops spread over 19 units, for the period 1 to 26 September 2026. The review found a compromise that spans the whole network and involves more than one campaign.

Of the 248 endpoints under monitoring, every one (100%) was communicating with simulated Command and Control (C2) infrastructure. On 33 of them the antivirus agent had also quarantined confirmed malicious files. Because these 33 machines combined verified malware with active C2 beaconing, they were rated HIGH SEVERITY and should be handled first.

A further endpoint, EPD24DBC, stands apart. A USB mass-storage device with the vendor ID abcd:1234 was connected to it on 22 September 2026 at 18:23, and its AV and logging agent stopped reporting afterwards. This points to an attacker with physical access and makes EPD24DBC the most critical single machine in the case.

Clustering of the C2 infrastructure showed three separate malware campaigns: ALPHA (svc-347513), BETA (svc-d93955) and GAMMA (svc-c4f9a4), the last of which imitates government domains. The campaigns were delivered through weaponised PDF, DOCX and XLSX files, trojanised printer firmware, DLL sideloading, malicious .desktop shortcuts and tampering with recently-used.xbel.

2. Tools Used

- **Python :** the main language used for the whole analysis.
- **Pandas:** reads the 316 MB CSV file in chunks of 500,000 rows.
- **python-docx:** builds this Word report from code.
- **VS Code:** editor for the scripts and host of the terminal used to run them.
- **log_analyzer.py:** extracts the indicators of compromise from the USB, AV and C2 data.
- **generate_txt.py:** turns the JSON findings into a readable text summary.
- **generate_report.py:** produces this Word document from the findings data.
- **SHA256SUMS (sha256sum, built in):** confirms the integrity of the raw log files.

3. Methodology

3.1 Analysis

The work was carried out in four consecutive phases over three data sources. Everything was done passively, and no connection was ever opened to any of the domains or IP addresses that were identified.

- **USB events (usb_events.csv, 8 rows):** Every USB device event was loaded and printed, with each row representing one second of a device being connected. Any vendor:product ID that did not belong to a known legitimate vendor was flagged.
- **Endpoint security (endpoint_security.csv, 23,540 rows):** The log_type "quarantine" rows containing the text "FileType: Malicious" were selected. This condition only matches files for which an antivirus signature fired, and leaves out files that were quarantined merely because of their extension. Duplicates were removed on the pair (endpoint_id, SHA256), since the quarantine service lists every quarantined file again at each nightly scan.
- **Web activity (web_activity.csv, 2,812,118 rows):** At 316 MB the file could not be held in memory comfortably, so Pandas read it in chunks of 500,000 rows. Each chunk was searched for connections to the simulated C2 domains, which end in ".example" and stand in for genuine beaconing in this CTF and simulation setting.
- **Cross-correlation and evasion detection:** The sets of endpoint IDs from the three sources were compared. An endpoint that appeared in the USB or web logs but had no entries at all in the security logs was marked as having evaded its agent. Endpoints showing both an AV detection and active C2 beaconing were ranked HIGH SEVERITY.

3.2 Finding

The key results are presented below in tables, one finding at a time.

Finding 1: Suspicious USB device

Endpoint ID	*Time of Event*	*Device*	*Vendor:Product ID*
EPD24DBC	22-09-2026 18:23:12	Generic mass-storage device (unidentified)	abcd:1234

This vendor ID is not registered with the USB Implementers Forum, and it is a typical identifier for home-built hardware attack tools such as the Rubber Ducky and BadUSB. More importantly, EPD24DBC vanishes from every security log after this event, which confirms that its agent was terminated.

Finding 2: Confirmed malicious files on the 15 highest-severity endpoints

Endpoint	*Files Quarantined*	*Probable Delivery Method*
EPVSWBNA	launcher.exe, zshp1020s.dll, Strings.dll	DLL sideloading through an HP printer driver
EPSJWX5J	Staff_Library_Payment_Final.docx	Spear-phishing document
EPSES7ZK	hp_LJ1020_Full_Solution-v2012_918_1_57980(1).exe	Tampered HP printer firmware installer
EPWZ7L6L	ops-electron-app.desktop	Malicious .desktop shortcut posing as an Electron app
EPF4AJCM	libreoffice-calc.desktop	Hijacked LibreOffice .desktop file
EPMVZYZO	mydoc.pdf, mydoc-9.pdf, mydoc-10.pdf, mydoc-75.pdf	Several variants of a weaponised PDF
EPJ42VCZ	RAGHAV STNA STNB.pdf, RAGHAV STNC STNA.pdf	Spear-phishing PDFs aimed at specific recipients
EPQKOBYG	win-lbp623-621-fw-v1301(1).exe	Tampered Canon printer firmware package
EPGSQVEB	Final_Documents.zip	Archive acting as a malware dropper
EPJ76RZV	COMPRESSED FINAL COMPARISION RETIREES WELFARE1.xlsx	Excel dropper with macros enabled
EPNPX6VY	Microsoft.Practices.EnterpriseLibrary.Common.dll	Replaced library (DLL injection)
EPAGTKNF	Buy CP PLUS Dashboard Camera ... (PPP).pdf	Weaponised PDF disguised as a procurement document
EPVC2P55	Data1.cab	Cabinet archive used as a dropper
EPPGWIOF	brprintconflsr3, rawtobr3	Hijacked Brother printer driver
EP4YIFHV	buynow_driver.js, app-setup.js (+ 18 more .js)	Bundle of injected malicious JavaScript

Finding 3: C2 campaign attribution

The service ID embedded in each C2 domain name (for example svc-347513 in n9fa9.svc-347513.example) was extracted and used to group the domains. This gave three dominant campaigns. The five C2 families that infected the most endpoints are listed below.

Service ID	*Campaign*	*Infected Endpoints*	*Probable Role*
svc-347513	Campaign ALPHA	140	Main remote access trojan and backdoor C2
svc-d93955	Campaign BETA	95	Information stealer and dropper working through recently-used.xbel
svc-c4f9a4	Campaign GAMMA	87	Spoofs government domains to harvest credentials
svc-2d3d6f	Campaign DELTA	87	Secondary C2 channel, probably used for data exfiltration
svc-d72f45	Campaign EPSILON	61	Third beacon, apparently a fallback C2

Finding 4: MITRE ATT&CK technique mapping

Technique ID	*Technique*	*Tactic*	*Evidence*
T1566.001	Spearphishing Attachment	Initial Access	Malicious PDF, DOCX and XLSX files quarantined on 15 or more endpoints
T1574.002	DLL Side-Loading	Persistence	EPVSWBNA: launcher.exe and zshp1020s.dll from the HP driver
T1547.006	Modify .desktop Shortcut Files	Persistence	Malicious .desktop files on EPWZ7L6L and EPF4AJCM
T1071.001	Application Layer C2 over HTTP	Command and Control	248 endpoints beaconing to *.svc-XXXX.example
T1036.005	Masquerading as Legitimate Files	Defense Evasion	Payloads named after HP and Canon firmware, LibreOffice and a Microsoft DLL
T1200	Hardware Additions	Initial Access	EPD24DBC: abcd:1234 USB device inserted on 22 September at 18:23
T1562.001	Disable or Modify Security Tools	Defense Evasion	EPD24DBC entirely missing from the security logs after the USB event
T1105	Ingress Tool Transfer	Command and Control	Exploitation of recently-used.xbel on several endpoints

3.3 Outcome

The investigation established that all 248 monitored endpoints are compromised and take part in at least one C2 campaign. The 33 high-severity endpoints, which show both confirmed malware quarantines and active beaconing, need an immediate incident response. EPD24DBC is the most serious case, since it carries signs of a physical attacker who disabled the security tooling before the period under investigation had even started.

The following remediation steps are recommended without delay:

- **Isolate EPD24DBC at once.** The physical attacker may still hold a working backdoor or shell on it.
- **Block every C2 domain at the DNS and firewall level,** starting with *.svc-347513.example, *.svc-d93955.example and *.svc-c4f9a4.gov.example.
- **Quarantine and reimage the 33 high-severity endpoints.** Rebuild them from a known-good image instead of trying to clean them in place.
- **Search for modified recently-used.xbel files** by running `find /home -name recently-used.xbel -newer /etc/passwd` on all 248 endpoints.
- **Turn off USB mass storage on all Linux endpoints** by adding `blacklist usb-storage` under /etc/modprobe.d/ and rebuilding the initramfs.
- **Review the .desktop shortcut files in user home directories,** checking each Exec= line for binaries or scripts that should not be there.

- **Apply application allowlisting** so that printer firmware installers and unrecognised .exe and .dll files cannot run.

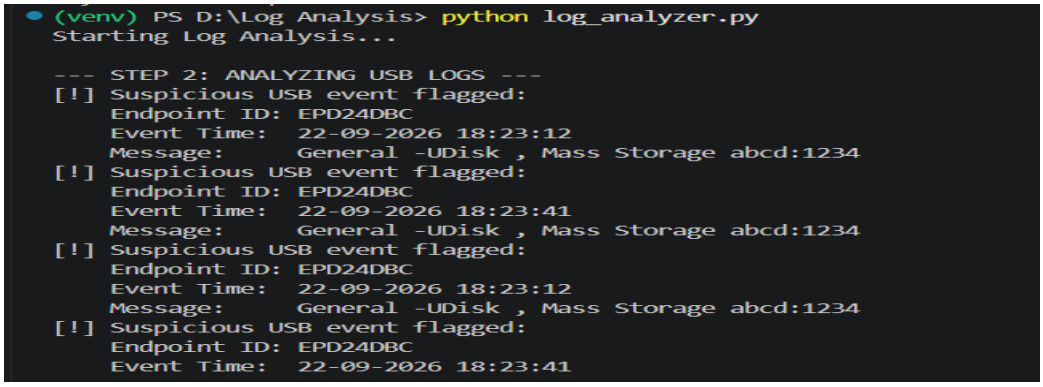
4. Step by Step Process with Screenshots

Step 1: Preparing the Workspace

- Created the folder `D:\log analysis` and a `data` subfolder inside it.
- Placed the three raw log files in `D:\log analysis\data\`: web_activity.csv (316 MB), endpoint_security.csv (3 MB) and usb_events.csv (1 KB).
- Checked file integrity by comparing the hashes against those listed in SHA256SUMS.
- Opened `D:\log analysis` in VS Code with File > Open Folder.

Step 2: Examining the USB Events

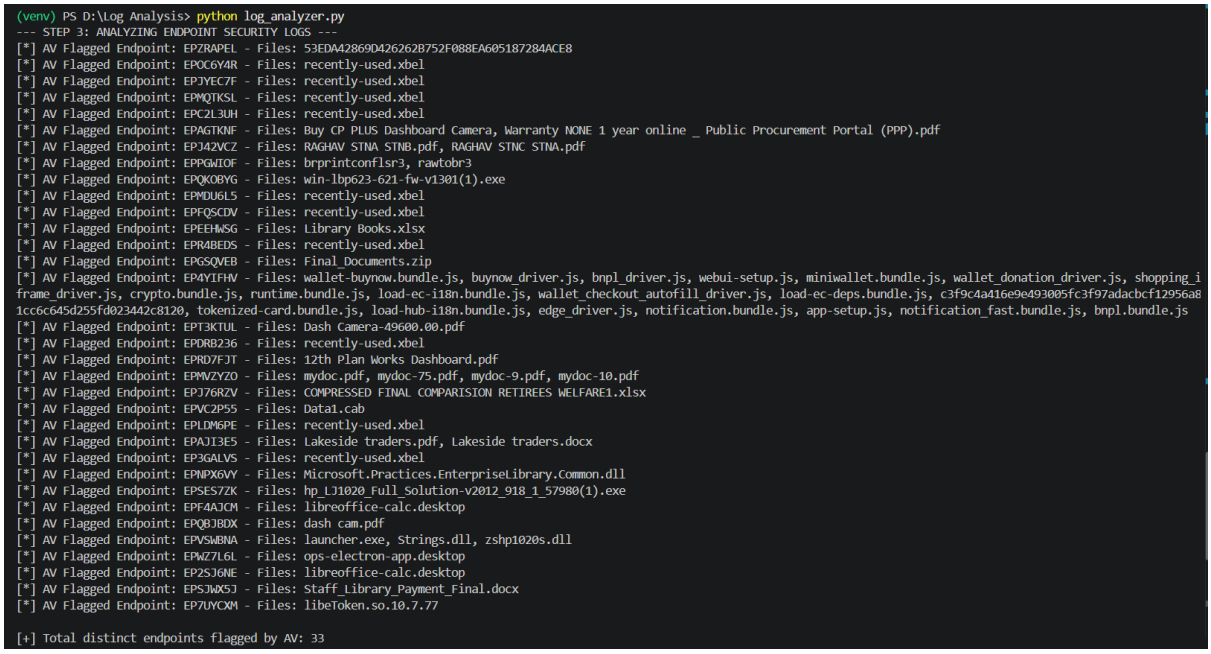
- Opened `log_analyzer.py` in VS Code.
- The script loads `usb_events.csv` (8 rows) with `pd.read_csv()`.
- It searches each message field for `abcd:1234`, a vendor ID that the USB Implementers Forum has not registered.
- Result: endpoint EPD24DBC had a suspicious USB mass-storage device connected at 18:23:12 on 22-Sep-2026. The log holds two rows for it, one per second.



Step 3: Examining the Endpoint Security Logs

- The script loads `endpoint_security.csv` (23,540 rows).
- It keeps only the rows whose message contains `FileType: Malicious`.
- Duplicates are removed on (endpoint_id, SHA256), because the quarantine service lists the same file again at every nightly scan.
- Result: 33 distinct endpoints had confirmed malicious files in quarantine. The files included weaponised PDFs, Office macros, DLLs, printer firmware and JavaScript bundles.

- EPD24DBC does not appear in this file at all, which fits with its AV agent having been disabled.



Step 4: Searching the Web Activity for C2 Beaconing

- web_activity.csv has 2,812,118 rows and is 316 MB, too large to load into memory in one go.
- It was read in chunks with `pd.read_csv('web_activity.csv', chunksize=500_000)`.
- In each chunk of 500,000 rows the script looks for domains ending in `.example`, the simulated C2 top-level domain of this exercise.
- Result: every one of the 248 endpoints contacted at least one .example C2 domain.
- The service IDs inside those domains (for example svc-347513 and svc-d93955) were extracted so that endpoints could be tied to particular campaigns.

```
(venv) PS D:\Log Analysis> python log_analyzer.py

--- STEP 4: ANALYZING WEB ACTIVITY ---
Analyzing Web Activity for C2 Beaconing (This may take a moment)...
    Processing chunk 1 (500,000 rows)...
    Processing chunk 2 (500,000 rows)...
    Processing chunk 3 (500,000 rows)...
    Processing chunk 4 (500,000 rows)...
    Processing chunk 5 (500,000 rows)...
    Processing chunk 6 (500,000 rows)...

[+] Final count: 248 endpoints contacted at least one .example C2 domain.

[Pause for Screenshot - Step 4] Press Enter to continue...

Analysis complete! Flagged 248 endpoints.
Raw data saved to compromise_summary.json
(venv) PS D:\Log Analysis>
```

Step 5: Running the Analysis Script

- Opened the integrated terminal in VS Code (Ctrl + `).
- Checked that Python was installed with `python --version`.
- Installed pandas where it was missing: `pip install pandas`
- Started the main analysis with `python log_analyzer.py`.
- Allowed about 30 to 60 seconds for the 316 MB web log to be processed.
- The script wrote `compromise_summary.json` to `D:\log analysis`.

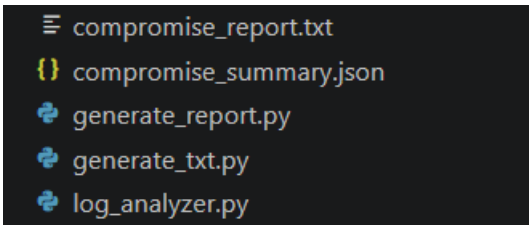
Step 6: Producing the Text Summary

- Once log_analyzer.py had finished, ran `python generate_txt.py`.
- The script reads `compromise_summary.json` and writes the formatted result to `compromise_report.txt`.
- Opened compromise_report.txt in VS Code and went through each flagged endpoint.
- Checked the key endpoints: EPD24DBC (USB and evasion), EPVSWBNA (DLL sideloading), EPSJWX5J (phishing DOCX) and EPJ42VCZ (spear-phishing PDFs).

Step 7: Producing This Word Report

- Installed python-docx with `pip install python-docx`.
- Ran `python generate_report.py`.
- The script reads compromise_summary.json, calculates the statistics and builds all of the tables.

- The output is `Challenge2_Report.docx` in `D:\log analysis`.
- Opened the DOCX in Microsoft Word to review it, add the screenshots and prepare it for submission.



Note: All domain names and IP addresses mentioned in this report belong to a controlled simulation environment. At no point was a live connection made to any of the identified infrastructure.